PROJECT REPORT

# Single-Cycle RISC-V (RV32I) Processor Core in Chisel

Design, implementation and simulation-based verification of a 32-bit single-cycle processor written in Chisel 3 / Scala



| | |
|---|---|
| Author | Abdul Rehman Ali |
| Project | Shadow (Scala-Chisel-Learning-Journey) |
| Architecture | RISC-V RV32I, single-cycle, Harvard-style memories |
| Language / tools | Chisel 3.4.3, Scala 2.12.13, ChiselTest 0.3.3 (Treadle), sbt 2.0.8, Java 17 |
| Date | October 2026 |

# Contents

**Contents** 2

**1. Introduction** 3

1.1 Goals . . . 3

1.2 Technology stack . . . 3

**2. Supported Instructions and Features** 3

**3. Project Structure** 4

**4. Architecture** 4

4.1 Module hierarchy . . . 4

4.2 Datapath . . . 5

Figure 2: Datapath block diagram . . . 6

4.3 How one instruction flows through the core . . . 7

**5. Module Reference** 7

**6. Control Logic** 8

6.1 Control signals per instruction type . . . 8

6.2 ALU operation codes . . . 8

6.3 Operand selection and next-PC selection . . . 9

**7. Memory System** 9

7.1 Instruction memory and program format . . . 9

7.2 Data memory . . . 9

**8. Testing and Verification** 9

8.1 Test setup . . . 9

8.2 Test program 1: multiplication by shift-and-add (Mul-9) . . . 10

8.3 Observed result . . . 10

8.4 Test program 2: store and load . . . 11

8.5 Viewing the waveform . . . 11

**9. Limitations and Review Notes** 11

9.1 Known limitations . . . 11

9.2 Notes from the code review . . . 12

**10. Future Work** 12

**11. Building and Running** 12

**12. Conclusion** 12

# 1. Introduction

This report documents a single-cycle implementation of the RISC-V RV32I base integer instruction set, written in Chisel, a hardware construction language embedded in Scala. In a single-cycle processor every instruction completes in exactly one clock cycle: the instruction is fetched, decoded, executed, optionally accesses data memory, and writes its result back to the register file before the next rising clock edge.

The design is split into small, single-purpose modules (program counter, instruction memory, decoder, controller, register file, ALU, and so on) that are wired together by a top-level **Core** module. The project is simulated with ChiselTest and the Treadle simulator. The test loads a hexadecimal program into instruction memory, runs the clock, and records a waveform (VCD) that can be inspected with GTKWave.

## 1.1 Goals

- Implement a clean, readable RV32I datapath and control unit in Chisel.
- Keep each hardware block in its own file and class so it can be understood and tested on its own.
- Load programs from a plain hex file so new test programs can be run without changing the hardware.
- Verify behaviour by simulation and waveform inspection.

## 1.2 Technology stack

| Component | Version / detail |
|---|---|
| Hardware language | Chisel 3.4.3 (Scala embedded DSL), with the `chisel3-plugin` |
| Scala | 2.12.13 |
| Build tool | sbt (`project/build.properties` pins 2.0.8) |
| Test framework | ChiselTest 0.3.3 with ScalaTest `FreeSpec`; Treadle simulator |
| Java | OpenJDK 17 |
| Waveform viewer | GTKWave (opens the generated `Core.vcd`) |

# 2. Supported Instructions and Features

The core implements the RV32I base integer instructions in the groups below. The decoder recognises nine opcode classes; everything else is treated as a no-operation (no register write, no memory access, PC advances by 4).

| Instruction class | Opcode | Instructions | Status |
|---|---|---|---|
| R-type | 0110011 | add, sub, sll, slt, sltu, xor, srl, sra, or, and | Supported |
| I-type (ALU) | 0010011 | addi, slti, sltiu, xori, ori, andi, slli, srli, srai | Supported |
| Load | 0000011 | lw | Word access only (see below) |
| Store | 0100011 | sw | Word access only (see below) |
| Branch (SB-type) | 1100011 | beq, bne, blt, bge, bltu, bgeu | Supported |
| JAL (UJ-type) | 1101111 | jal | Supported |
| JALR | 1100111 | jalr | Supported |

| Instruction class | Opcode | Instructions | Status |
|---|---|---|---|
| LUI | 0110111 | lui | Supported (see review notes, Section 9) |
| AUIPC | 0010111 | auipc | Decoded; see review notes, Section 9 |

**Not supported:** byte and half-word memory accesses (`lb, lbu, lh, lhu, sb, sh`). The `funct3` field of load and store instructions is ignored, so these instructions behave as word accesses. System instructions (`ecall, ebreak, fence`), CSR instructions and all standard extensions (M, A, F, C, ...) are also outside the scope of this core.

# 3. Project Structure

```
Shadow/
├── build.sbt                       Scala/Chisel dependencies and compiler options
├── project/build.properties        sbt version (2.0.8)
├── README.md                       Setup and run instructions
├── src/
│   ├── main/scala/processor/       Hardware source (package `processor`)
│   │   ├── Core.scala              Top level: instantiates and wires every block
│   │   ├── PC.scala                Program counter
│   │   ├── Memory.scala            Instruction memory (loads hex file)
│   │   ├── InstrDecoder.scala      Splits the instruction into fields
│   │   ├── Controller.scala        Wrapper: TypeController + SignalController
│   │   ├── TypeController.scala    Opcode -> one-hot instruction type
│   │   ├── SignalController.scala  Instruction type -> datapath control signals
│   │   ├── ALUControl.scala        ALUOp + funct3/funct7 -> 5-bit ALU operation
│   │   ├── PriEncoder.scala        8-to-3 priority encoder used by ALUControl
│   │   ├── AluOp.scala             ALU operation codes
│   │   ├── ALU.scala               Arithmetic/logic/compare unit
│   │   ├── Branch.scala            Branch-taken decision
│   │   ├── RegisterFile.scala      32 x 32-bit registers
│   │   ├── ImmGen.scala            Immediate generator
│   │   ├── JALRAdder.scala         JALR target adder
│   │   ├── DataMem.scala           Data memory
│   │   ├── Opcode.scala            Opcode constants
│   │   └── IOInterface.scala       Shared Bundle definitions between modules
│   └── test/
│       ├── scala/CoreTest.scala    ChiselTest simulation of the whole core
│       └── resources/              Hex programs: Mul-9.txt, test.txt
└── test_run_dir/                   Generated: FIRRTL and Core.vcd waveform
```

# 4. Architecture

## 4.1 Module hierarchy

**Core** is the only module with children at the top of the hierarchy. The Controller and ALU Control each contain helper modules. Blue modules are datapath blocks, yellow modules form the control unit and green modules are the ALU path.

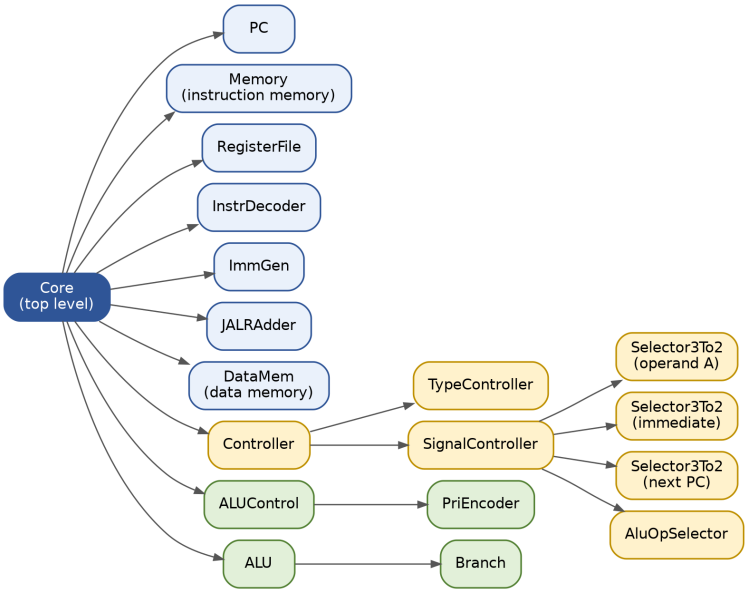



*Figure 1: Module hierarchy of the processor (generated from the Chisel source).*

## 4.2 Datapath

The datapath is shown on the next page (Figure 2). Reading it from left to right: the **PC** addresses the **Instruction Memory**; the **Instruction Decoder** splits the 32-bit instruction into the opcode, register indices and function fields; the **Register File** returns two operands; two multiplexers choose the ALU inputs (register, PC, PC+4 or immediate); the **ALU** computes the result, which is either written back, used as a data-memory address, or used for a branch decision; finally the **Next-PC logic** chooses the address of the following instruction.

## Figure 2: Datapath block diagram

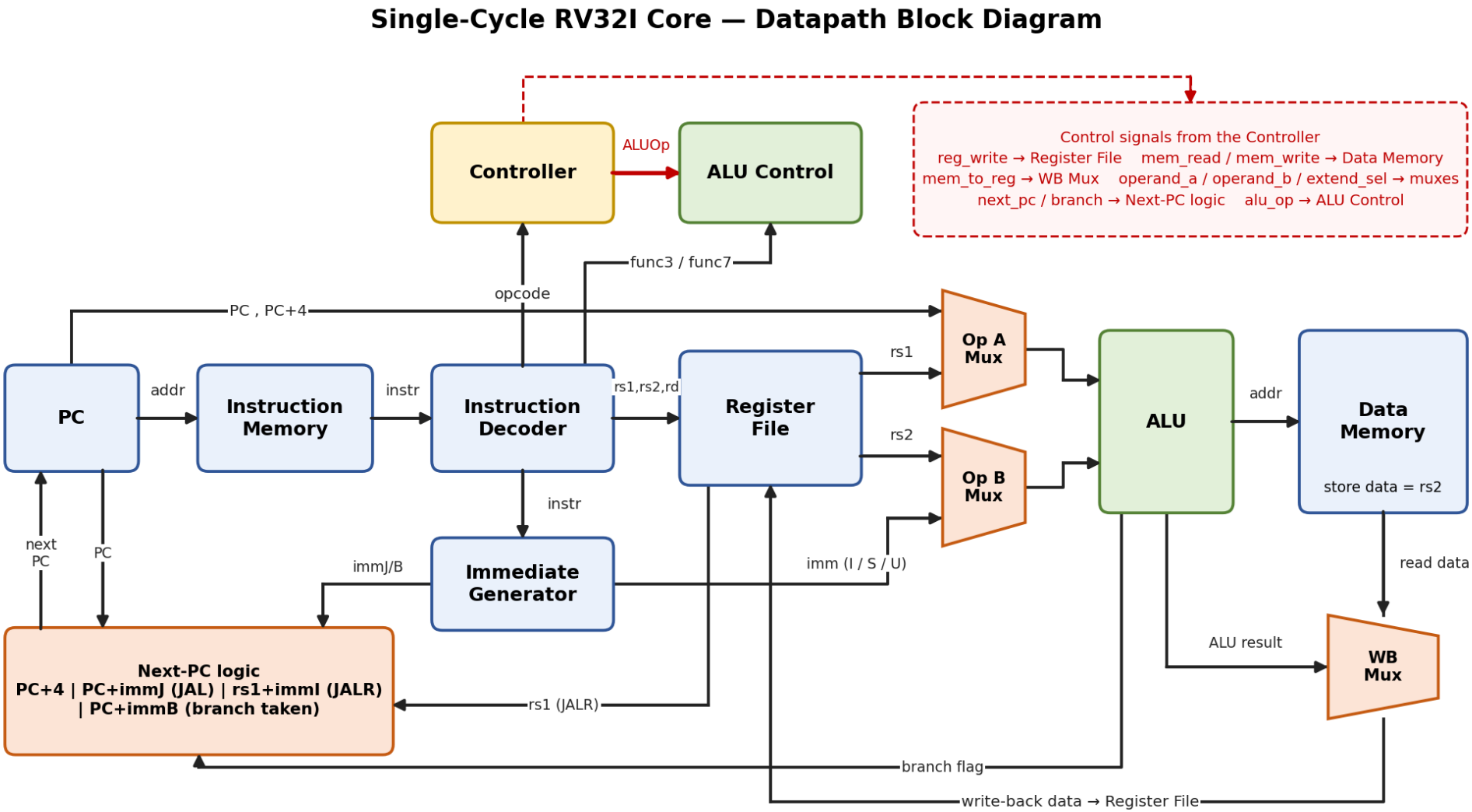



*Figure 2: Single-cycle datapath. Black arrows are data, the red dashed box summarises the control signals produced by the Controller.*

### 4.3 How one instruction flows through the core

All of the steps below happen combinationally within one clock cycle. The PC register and the register file are the only state elements that change at the clock edge (together with the data memory on a store).

| Stage | What happens |
|---|---|
| Fetch | The PC register outputs the current address. Instruction memory is indexed with address bits [11:2], so it holds 1024 words (4 KiB), and returns the 32-bit instruction. |
| Decode | InstrDecoder extracts opcode [6:0], rd [11:7], funct3 [14:12], rs1 [19:15], rs2 [24:20] and funct7 bit 30. ImmGen builds the I, S, SB, U and UJ immediates. TypeController turns the opcode into one-hot instruction-type flags. |
| Control | SignalController derives reg_write, mem_read, mem_write, mem_to_reg, branch, operand_a, operand_b, extend_sel, next_pc and a 3-bit ALUOp. ALUControl combines ALUOp with funct3/funct7 into a 5-bit ALU operation. |
| Execute | Operand A is rs1, the PC, or PC+4. Operand B is rs2 or an immediate. The ALU performs the operation; for branches it produces a 0/1 comparison result that the Branch block turns into branch-taken. |
| Memory | For loads and stores the ALU result is the data-memory address. Stores write rs2; loads read the addressed word. |
| Write-back | mem_to_reg selects the memory data or the ALU result, and reg_write enables writing it into register rd (x0 is never written). |
| Next PC | next_pc selects PC+4, PC + immUJ (JAL), (rs1 + immI) with bit 0 cleared (JALR), or for a taken branch PC + immSB. |

# 5. Module Reference

Each module is a Chisel `Module` in package `processor`. The table summarises its purpose and behaviour as implemented in the source.

| Module | Role and behaviour |
|---|---|
| **Core**<br>Core.scala | Top level. Takes the hex program path `initFile: String` as a constructor parameter. Instantiates all blocks and wires them together. Its only output, `io.out` (32 bits), shows the instruction currently fetched from instruction memory, which makes the program visible in the waveform. |
| **PC**<br>PC.scala | 32-bit register, reset value 0, loaded with `pc_in` every cycle. Outputs `addr` (current PC) and `pc_out` (PC + 4). |
| **Memory**<br>Memory.scala | Instruction memory: `Mem(1024, UInt(32.W))` read combinationally at `addr(11,2)`. Initialised from the hex file with `loadMemoryFromFile(mem, initFile)`. |
| **InstrDecoder**<br>InstrDecoder.scala | Pure wiring: outputs the opcode, rs1, rs2, rd, funct3 and funct7 (bit 30 only, which is all that RV32I needs to distinguish add/sub and srl/sra). |
| **RegisterFile**<br>RegisterFile.scala | 32 signed 32-bit registers. Two combinational read ports (reading x0 always returns 0). One write port, written at the clock edge when `reg_write` is high and rd is not x0. |
| **ImmGen**<br>ImmGen.scala | Produces five sign-extended immediates from the instruction: I, S, SB (branch), U and UJ (jump). |
| **Controller**<br>Controller.scala | Wrapper that connects **TypeController** (opcode to one-hot instruction type: R, I, LOAD, S, SB, UJ, JALR, LUI, AUIPC) to **SignalController** (instruction type to control signals). |

| Module | Role and behaviour |
|---|---|
| **SignalController**<br>SignalController.scala | Generates the datapath control signals using three `Selector3To2` encoders (operand A, immediate type, next PC) and an `Alu0pSelector` (3-bit ALUOp). See Section 6. |
| **ALUControl**<br>ALUControl.scala | Uses a `PriEncoder` to classify the ALUOp/funct7 combination and builds the 5-bit ALU operation as {2-bit group, funct3}. See Section 6.2. |
| **ALU**<br>ALU.scala | Signed 32-bit ALU. Implements add, sub, sll, slt, sltu, xor, srl, sra, or, and, the six branch comparisons (returns 0/1) and copy-A (used by JAL/JALR to pass PC+4 through). Contains a `Branch` block. |
| **Branch**<br>Branch.scala | `br_taken = alu_result(0) AND (alu_op(4,3) == 2)`, i.e. the comparison result counts only for branch operations (codes 16 to 23). |
| **JALRAdder**<br>JALRAdder.scala | Computes `(rs1 + immI) & 0xFFFFFFFE`, the JALR target address. |
| **DataMem**<br>DataMem.scala | Data memory: `Mem(32, SInt(32.W))`. Writes rs2 when `mem_write` is set; reads when `mem_read` is set. See review notes in Section 9. |

# 6. Control Logic

## 6.1 Control signals per instruction type

The table is derived directly from the logic in `TypeController` and `SignalController`. Operand A: 0 = rs1, 1 = PC, 2 = PC+4, 3 = rs1 (read port 1). Operand B: 0 = rs2, 1 = immediate chosen by extend_sel (0 = I, 1 = S, 2 = U). Next PC: 0 = PC+4, 1 = JAL target, 2 = JALR target, 3 = branch (taken: PC + immSB, else PC+4).

| Type | reg_write | mem_read | mem_write | mem_to_reg | branch | op_A | op_B | ext | next_pc | ALUOp |
|---|---|---|---|---|---|---|---|---|---|---|
| R | 1 | 0 | 0 | 0 | 0 | 0 | 0 | - | 0 | 000 |
| I | 1 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 001 |
| LOAD | 1 | 1 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 100 |
| S | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 | 0 | 101 |
| SB | 0 | 0 | 0 | 0 | 1 | 0 | 0 | - | 3 | 010 |
| JAL (UJ) | 1 | 0 | 0 | 0 | 0 | 2 | 0 | - | 1 | 011 |
| JALR | 1 | 0 | 0 | 0 | 0 | 2 | 0 | - | 2 | 011 |
| LUI | 1 | 0 | 0 | 0 | 0 | 3 | 1 | 2 | 0 | 110 |
| AUIPC | 0 * | 0 | 0 | 0 | 0 | 1 | 1 | 2 | 0 | 111 |

** As implemented, `reg_write` is not asserted for AUIPC; see Section 9.*

## 6.2 ALU operation codes

`ALUControl` produces a 5-bit code. For arithmetic and logic operations the upper bits select the group and the lower three bits are `funct3`: group 0 = normal operations, group 1 = the "alternate" operations selected by funct7 (sub, sra), group 2 = branch comparisons. ALUOp 011 (JAL/JALR) selects code 31 (copy A); ALUOps that match no other pattern (loads, stores, LUI, AUIPC) select ADD.

| Code | Operation | Code | Operation | Code | Operation |
|---|---|---|---|---|---|
| 0 | ADD | 8 | SUB | 16 | BEQ |
| 1 | SLL | 13 | SRA | 17 | BNE |

| Code | Operation | Code | Operation | Code | Operation |
|---|---|---|---|---|---|
| 2 | SLT | | | 20 | BLT |
| 3 | SLTU | | | 21 | BGE |
| 4 | XOR | | | 22 | BLTU |
| 5 | SRL | | | 23 | BGEU |
| 6 | OR | | | 31 | COPY_A |
| 7 | AND | | | | |

### 6.3 Operand selection and next-PC selection

Operand A selects between rs1 (R, I, load, store, branch), the PC (AUIPC) and PC+4 (JAL, JALR, so that the link address is written to rd). Operand B selects rs2 or one of the immediates. The next-PC multiplexer in `Core` chooses the sequential address, the JAL target (PC + immUJ), the JALR target from `JALRAdder`, or for a branch the target PC + immSB when `alu.branch & controller.branch` is true.

## 7. Memory System

### 7.1 Instruction memory and program format

Instruction memory holds 1024 32-bit words and is initialised from a text file passed to the `Core` constructor. The file is plain hexadecimal with **one 32-bit instruction per line**, no `0x` prefix. For example, the first lines of `Mul-9.txt`:

```
00900513
00a00593
00100693
00000713
...
```

Addresses are byte addresses: instruction *n* is at address 4*n*. Locations beyond the end of the program are zero. A word of zero has opcode 0, which the controller treats as a no-operation.

### 7.2 Data memory

Data memory is a 32-word array of signed 32-bit values with a combinational read and a synchronous write. Only full-word accesses exist; there is no byte or half-word support. See Section 9 for review notes on its address mapping.

## 8. Testing and Verification

### 8.1 Test setup

`CoreTest` (in `src/test/scala`) instantiates `Core` with the path of a hex program, then advances the clock by 100 cycles. Simulation runs on Treadle through ChiselTest, and a waveform is written to `test_run_dir/Testing_Core/Core.vcd`, together with the FIRRTL files for the design. Verification is done by inspecting the waveform (or by adding expectations to the test).

```
test(new Core("src/test/resources/Mul-9.txt")) { dut =>
  dut.clock.step(100)
}
```

To run a different program, change the path to another hex file such as `src/test/resources/test.txt`. The commands to build and run the test are in **README.md**.

## 8.2 Test program 1: multiplication by shift-and-add (Mul-9)

`Mul-9.txt` multiplies 9 by 10 using the shift-and-add method: it tests the low bit of the multiplier (a0), adds the multiplicand (a1) to the accumulator (a4) when that bit is 1, then shifts the multiplier right and the multiplicand left until the multiplier reaches zero. It exercises I-type ALU instructions, shifts, and-immediate, R-type add, forward and backward branches and `jal`.

| Addr | Hex | Assembly | Meaning |
|---|---|---|---|
| 0x00 | 00900513 | addi a0, zero, 9 | a0 = 9 (multiplier) |
| 0x04 | 00a00593 | addi a1, zero, 10 | a1 = 10 (multiplicand) |
| 0x08 | 00100693 | addi a3, zero, 1 | a3 = 1 (constant) |
| 0x0c | 00000713 | addi a4, zero, 0 | a4 = 0 (accumulator) |
| 0x10 | 00157613 | andi a2, a0, 1 | a2 = lowest bit of a0 |
| 0x14 | 00d60a63 | beq a2, a3, 0x28 | if bit is 1, go add |
| 0x18 | 00155513 | srli a0, a0, 1 | a0 >>= 1 |
| 0x1c | 00050a63 | beq a0, zero, 0x30 | multiplier is 0: finish |
| 0x20 | 00159593 | slli a1, a1, 1 | a1 <<= 1 |
| 0x24 | fedff06f | jal zero, 0x10 | loop |
| 0x28 | 00b70733 | add a4, a4, a1 | a4 += a1 |
| 0x2c | fe0006e3 | beq zero, zero, 0x18 | jump back (always taken) |
| 0x30 | 00a50513 | addi a0, a0, 10 | end of program |

## 8.3 Observed result

The waveform produced by the project's own test run (Core.vcd, 100 clock cycles) was analysed. The PC sequence starts `0x00, 0x04, 0x08, 0x0c, 0x10, 0x14, 0x28, 0x2c, 0x18, 0x1c, 0x20, 0x24, 0x10, ...`, which is exactly what the program should do (the first multiplier bit is 1, so the add at 0x28 runs). The same program was also executed on an independent RV32I reference interpreter and produced the identical PC sequence and final register values.

| Register | ABI name | Final value | Role |
|---|---|---|---|
| x10 | a0 | 10 | Multiplier (reaches 0, then the final addi sets it to 10) |
| x11 | a1 | 80 | Multiplicand after shifting: 10 << 3 |
| x12 | a2 | 1 | Last tested multiplier bit |
| x13 | a3 | 1 | Constant 1 |
| **x14** | **a4** | **90** | **Result: 9 × 10 = 90** |

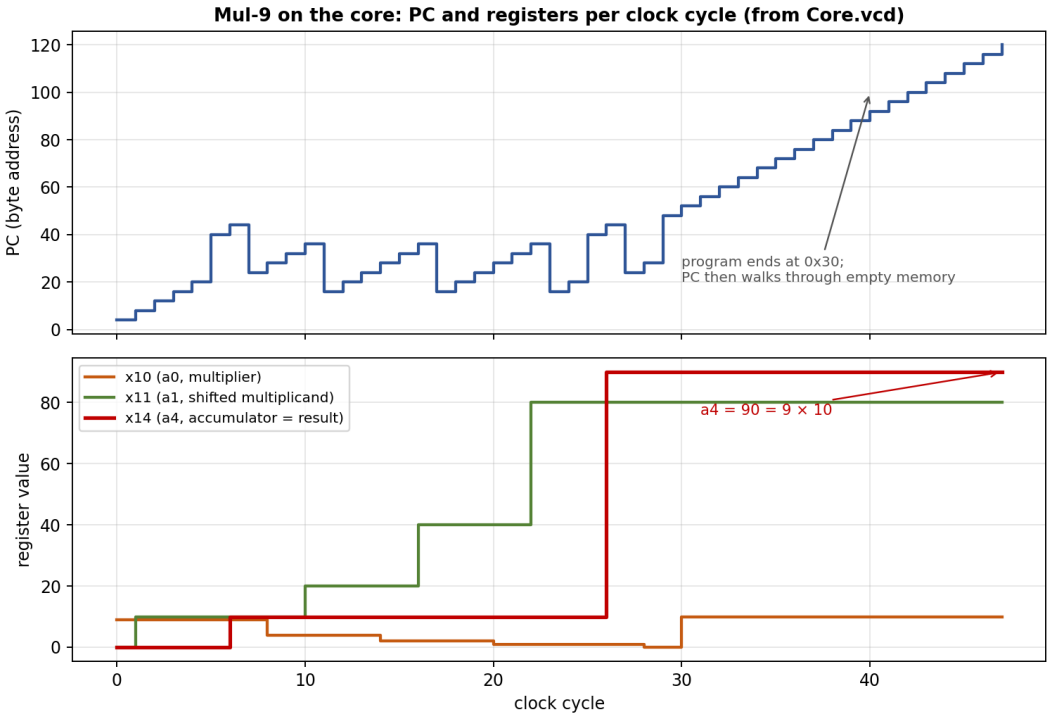



*Figure 3: PC and register values per clock cycle, extracted from Core.vcd. The accumulator a4 receives 10 and then 80, ending at 90. After the last instruction the PC keeps advancing through empty (zero) memory.*

## 8.4 Test program 2: store and load

`test.txt` is a short program that builds a constant, stores it and loads it back:

| Addr | Hex | Assembly |
|---|---|---|
| 0x00 | 00258537 | lui a0, 0x258 |
| 0x04 | cd550513 | addi a0, a0, -811 |
| 0x08 | 00a02023 | sw a0, 0(zero) |
| 0x0c | 00001583 | load into a1 from 0(zero) (encoded as lh) |

The last instruction is encoded as `lh`; since the core ignores funct3, it performs a word load. This program is useful for checking the store/load path (see the review notes in Section 9 before relying on it).

## 8.5 Viewing the waveform

Open the VCD in GTKWave and add the signals `Core.io_out` (current instruction), `Core.pc.io_addr` (PC) and `Core.reg_file.RegFile_*` (the 32 registers) to the waveform view. Each clock cycle is 10 ns in the generated file.

# 9. Limitations and Review Notes

## 9.1 Known limitations

- Only word-sized loads and stores (`lw`, `sw`). `lb, lbu, lh, lhu, sb, sh` are not supported.
- No system/CSR instructions and no M, A, F or C extension.
- Programs have no halt mechanism: after the last instruction the PC continues through zero-filled memory.

- `CoreTest` only runs the clock and records a waveform; it contains no automatic pass/fail checks.

### 9.2 Notes from the code review

While preparing this documentation the source was read closely. The points below follow directly from the code and are listed so they can be checked and, where appropriate, fixed. They are recommended follow-ups rather than a verified test failure.

| # | Where | Observation | Suggested follow-up |
|---|---|---|---|
| 1 | SignalController | `reg_write` is the OR of the R, I, load, LUI, JAL and JALR flags; the AUIPC flag is not included, so AUIPC would not write rd. | Add `io.in.auipc_type` to the reg_write expression and test with an auipc program. |
| 2 | Core (operand A) | For LUI the operand-A select is 3, which chooses register read port 1. The rs1 field of a LUI instruction is part of the immediate, so the result is rs1 + imm instead of 0 + imm. It is correct only when that register happens to be 0 (true in the provided programs). | Select zero (or copy B) as operand A for LUI. |
| 3 | DataMem | `io.data := DontCare` appears after the `when` block. Chisel uses last-connect semantics, so it overrides the conditional read and load data may not reach write-back. | Remove the DontCare line (or move it before the when block) and run an sw followed by lw test. |
| 4 | DataMem | The address input is `log2Ceil(32)` = 5 bits wide, so the low five bits of the byte address index the 32 words directly (not address bits [6:2]). | Index with a word address, e.g. `addr(6,2)`, if standard word alignment is wanted. |
| 5 | RegisterFile | The registers are `Reg(Vec(...))` with no reset value. The simulator starts them at 0, but real hardware would not. | Use `RegInit(VecInit(Seq.fill(32)(0.S(32.W))))`. |
| 6 | Conditionsf3.scala | The file has no package declaration and its constants are not used elsewhere. | Add `package processor` or remove the file. |

# 10. Future Work

- Resolve the review notes in Section 9 and add self-checking assertions to `CoreTest`.
- Add byte and half-word loads/stores (lb, lbu, lh, lhu, sb, sh) with sign/zero extension.
- Run the official RISC-V compliance tests (riscv-tests / riscv-arch-test) against the core.
- Generate Verilog from the design and test on an FPGA or with Verilator.
- Extend the ISA (M extension for multiply/divide) and, later, a pipelined version with hazard handling.

# 11. Building and Running

Full step-by-step instructions (installing Java, sbt, GTKWave and Verilator, cloning, running the test and viewing the waveform) are in **README.md** in the project root. In short, after installing the tools:

```
git clone <your-repository-url>
cd Shadow
sbt test
```

The first run downloads the Scala and Chisel dependencies and can take several minutes.

# 12. Conclusion

The project delivers a modular single-cycle RV32I core in Chisel with R-type, I-type, branch, JAL, JALR, LUI/AUIPC-class instructions and word-sized loads and stores. A shift-and-add multiplication program runs correctly on the core in simulation, producing 9 × 10 = 90 in register a4, which validates the fetch, decode, ALU, register file, branch and jump paths together. The review notes and future-work list above identify the next steps needed to make the design complete and self-checking.